

Real-time Updates

問題在哪裡

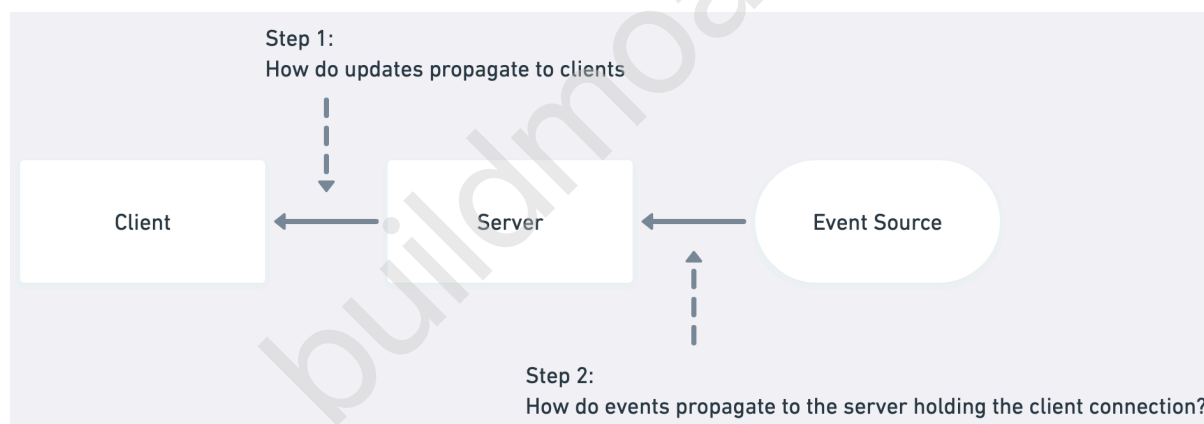
想像一個像 Google Docs 這樣的協作文件編輯器。當某個用戶打了一個字，所有其他在看這份文件的用戶需要在幾毫秒內看到這個改動。在這類應用中，你不可能讓每個用戶每隔幾毫秒就輪詢一次伺服器來取得更新，那樣會把你的基礎設施打垮。

核心挑戰在於在客戶端和伺服器之間建立高效、持久的通訊頻道。標準 HTTP 遵循請求-回應模型：客戶端要求資料，伺服器回應，然後連線關閉。這對傳統的網頁瀏覽很好用，但當你需要伺服器主動把更新推送給客戶端時就行不通了。

解法的架構：兩個 Hop

當系統需要即時更新、推播通知等功能，解法需要處理兩個截然不同的問題：

- **第一個 hop**：如何把更新從伺服器送達客戶端？
- **第二個 hop**：如何把更新從事件來源傳播到伺服器？



我們會分別拆解每個 hop，因為它們涉及不同的取舍，但兩者共同組成完整的解法。

第一個 Hop：客戶端-伺服器連線協定

第一個 hop 是在客戶端和伺服器之間建立高效的通訊頻道。在深入各種即時通訊協定之前，先了解一些網路基礎知識很有幫助，因為這類問題從根本來說就是網路問題。

網路基礎知識

網路是建立在分層架構上的（所謂的「OSI 模型」），這讓我們這些坐在最上層的應用程式開發者的世界大幅簡化。在系統設計面試中最常出現的有三個關鍵層次：

網路層 (Layer 3)：這一層是 IP 協定，負責路由和定址，把資料切成封包、在網路間轉發，並盡力把封包送到目的 IP。但不保證封包一定送達，可能會遺失、重複或亂序。

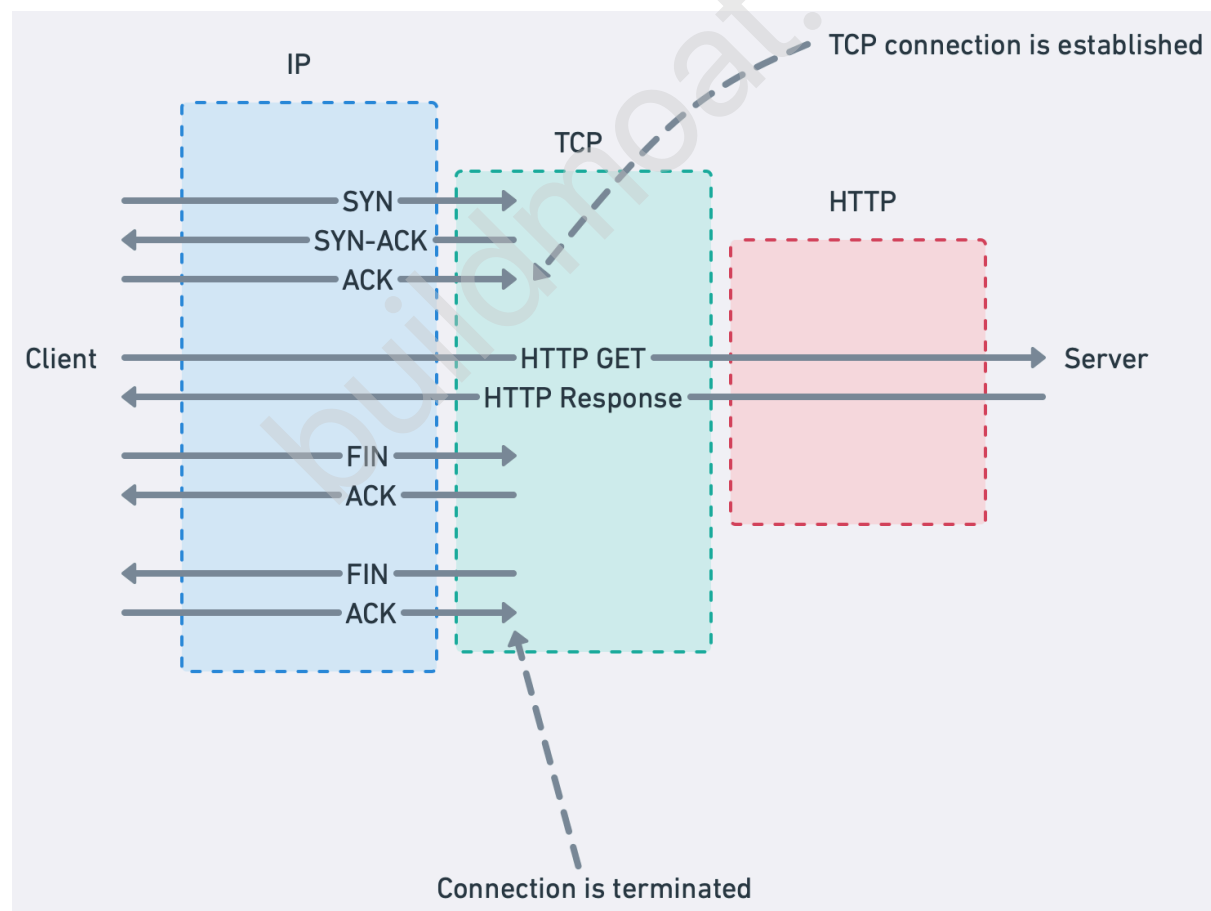
傳輸層 (Layer 4)：這一層有 TCP 和 UDP：

- **TCP** 是面向連線的協定：送資料之前必須先和對方建立連線。建立後，它確保資料正確且按序送達。這個保證很棒，但也意味著 TCP 連線需要時間建立、資源維護和頻寬消耗。
- **UDP** 是無連線的協定：不需要任何前置設定，直接把資料發出去。不保證正確送達或按序到達，比較像是盡力而為的傳輸。

應用層 (Layer 7)：最頂層是應用層協定，例如 DNS、HTTP、WebSocket、WebRTC。這些協定建立在 TCP 之上，為不同類型的資料提供抽象層。

HTTP 請求的完整旅程

當你在瀏覽器輸入 URL，一連串網路協定就開始運作。以下是一個簡單 HTTP 請求的流程：



1. **DNS 解析**：把域名（例如 [google.com](https://www.google.com)）解析成 IP 地址
2. **TCP 三次握手**：

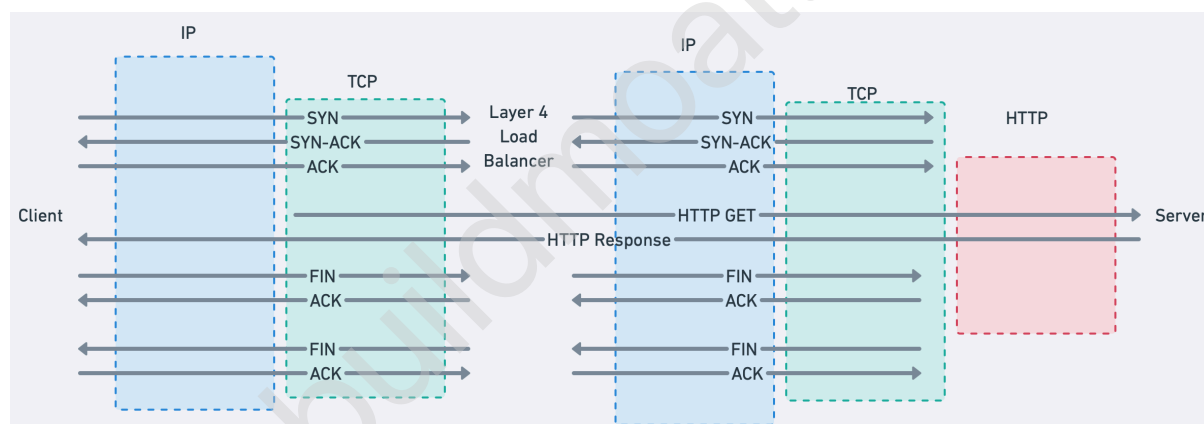
- SYN：客戶端送出同步封包請求連線
 - SYN-ACK：伺服器回傳確認
 - ACK：客戶端確認，連線建立完成
3. **HTTP 請求**：連線建立後，客戶端發送 HTTP GET
 4. **伺服器處理並回應**
 5. **TCP 四次揮手**：資料傳輸完成後雙方關閉連線

這裡有兩個對系統設計特別重要的觀察：第一，每一次客戶端與伺服器之間的來回都增加延遲，包括在送出應用資料之前建立連線的那幾次；第二，TCP 連線本身是一種需要雙方維護的狀態，如果不用 HTTP keep-alive，每個請求都要重複這個建立過程。

負載平衡器：L4 vs. L7

真實世界的系統通常有多台伺服器在負載平衡器後面協同工作。理解 L4 和 L7 負載平衡器的差異，對後面要做的設計決策很重要。

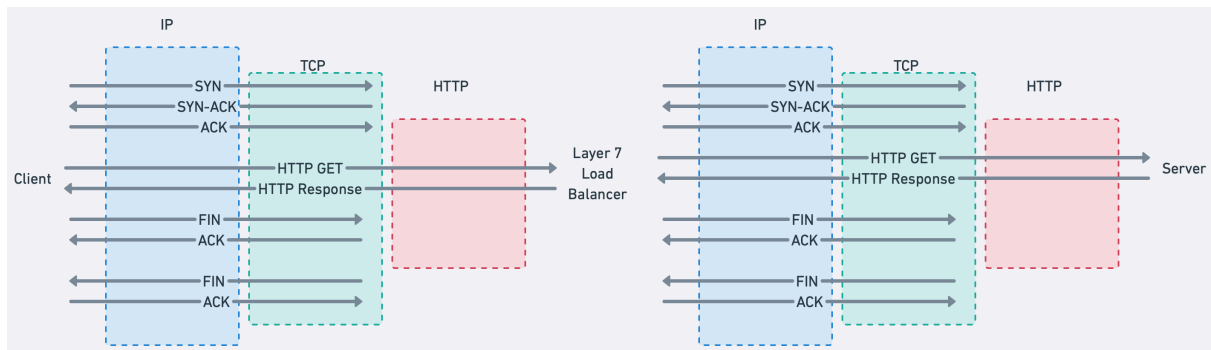
Layer 4 負載平衡器在傳輸層（TCP/UDP）運作，根據 IP 地址和 port 等網路資訊做路由決策，不看封包內容。你可以把它想成：一旦選定了後端伺服器，TCP 連線就直接在客戶端和那台伺服器之間建立。



L4 負載平衡器的特點：

- 維護客戶端和伺服器之間的持久 TCP 連線
- 因為不需要深度封包檢查，速度快、效率高
- 無法根據應用層資料做路由決策
- 天然適合需要持久連線的協定，例如 WebSocket

Layer 7 負載平衡器在應用層運作，理解 HTTP 等協定，能檢查每個請求的實際內容並做更智慧的路由決策——例如根據 URL、header、cookie 路由，或讓同一個用戶的請求永遠打到同一台伺服器。



L7 負載平衡器的特點：

- 終止進來的連線，再對後端伺服器建立新連線
- 更靈活，但 CPU 密集度更高
- 更適合基於 HTTP 的流量

在討論即時功能時，選擇 L4 還是 L7 常常是面試中的討論點。一般原則是：L4 更適合 WebSocket 連線，L7 對長輪詢這類 HTTP 方案提供更多靈活性。

好，基礎知識說完了，讓我們進入各種即時更新方案的正題。以下用一個聊天應用為例，用戶需要即時收到發到聊天室的新訊息。

簡單輪詢（Simple Polling）：基準方案

最簡單的做法是讓客戶端定期向伺服器輪詢更新。這在技術上算不上「即時」，但它是個很好的出發點，也提供了和其他方法對比的基準。

很多面試題其實不需要真正的即時更新。認真思考產品本身，問問自己低頻更新（例如每 2 到 5 秒一次）是否能滿足需求。如果可以，直接提議輪詢方案。提出一個稍微不完美的解法，遠比無法實作複雜方案要好。

運作方式非常簡單，比如客戶端以固定間隔發請求，伺服器回傳當前狀態。在聊天 App 中，我們就是一直輪詢「我還沒收到哪些訊息？」。

```
async function poll() {  
  const response = await fetch('/api/updates');  
  const data = await response.json();  
  processData(data);  
}  
  
// 每 2 秒輪詢一次  
setInterval(poll, 2000);
```

優點

- 實作簡單
- 無狀態
- 不需要特殊基礎設施
- 可以和任何標準網路基礎設施一起運作
- 不需要花太多時間解釋

最後一點被嚴重低估。如果你的問題核心不是即時更新，就用簡單輪詢，把精力和面試時間留給真正重要的部分。

缺點

延遲高於其他方法，更新可能延遲長達輪詢間隔加上請求處理時間

- 更新頻率有限
- 頻寬用量較高
- 大量客戶端時，反覆建立新連線的資源消耗可觀

面試中怎麼說

說明你在輪詢和其他方法之間做的取捨。一個好的表達是給自己留後路：「我先用簡單輪詢，這樣可以專注在 X，如果後來需要更即時的方案，再切換也不難。」

應對面試官對輪詢的最常見反對意見，例如「太慢了」或「不夠有效率」，可以討論如何利用 HTTP keep-alive 減少 overhead：把 keep-alive 的時間設得比輪詢間隔更長，這樣大多數情況下 TCP 連線只需要建立一次。

長輪詢（Long Polling）：簡單的近即時方案

長輪詢是達到接近即時更新最簡單的升級方案。它建立在標準 HTTP 之上，容易實作和擴展。

概念也很簡單：客戶端發出請求，伺服器把請求掛著，直到有新資料才回應。就像伺服器「處理很慢」，但其實是在等待。伺服器回應並結束 HTTP 請求後，客戶端立刻發出新請求。如果等了很久都沒有新資料，可以回傳空回應讓客戶端重新請求。

在聊天 App 中，我們持續請求「下一條訊息」。如果沒有新訊息，伺服器就把請求掛著，直到有新訊息才回傳。收到訊息後，立刻再發一個請求等下一條。

```
async function longPoll() {
  while (true) {
    try {
      const response = await fetch('/api/updates');
      const data = await response.json();
      processData(data);
    } catch (error) {
      console.error(error);
      // 出錯後稍等再重試
      await new Promise(resolve => setTimeout(resolve, 1000))
    }
  }
}
```

長輪詢的簡單背後隱藏了一個對高頻更新的重要取舍。假設客戶端和伺服器之間的延遲是 100ms，如果有兩個更新相隔 10ms 發生，我們在第一個更新發生後 100ms 才收到它（100ms 網路延遲），但第二個更新可能在發生後 290ms 才收到（90ms 等第一個回應結束 + 100ms 新請求 + 100ms 回應）。

優點

- 建立在標準 HTTP 上，在 HTTP 能運作的地方都能用
- 容易實作
- 不需要特殊基礎設施
- 伺服器端無狀態

缺點

- 延遲高於其他替代方案
- 更多 HTTP overhead
- 大量客戶端時資源消耗較大
- 高頻更新時不適合（原因如上）
- 因為請求可能掛很久，監控起來比較麻煩
- 瀏覽器對同一個域名的並發連線數有限制

什麼時候適合用

長輪詢是一個很好的「近即時」方案，適合更新不頻繁且偏好簡單實作的場景。它也非常適合需要知道「一個長時間非同步流程什麼時候完成」的場景，例如付款處理，我們長輪詢付款狀態，一旦完成立刻顯示成功頁面。

面試中怎麼說

因為長輪詢使用現有的 HTTP 基礎設施，不需要討論太多額外的基礎設施。但要記得說清楚你的 timeout 設定：你不希望負載平衡器在 30 秒後就掛斷連線，但你的長輪詢伺服器願意等 60 秒。常見的輪詢間隔是 15 到 30 秒，可以讓這個問題不那麼麻煩。

Server-Sent Events (SSE)：單向高效推送

SSE 是長輪詢的進化版，讓伺服器可以把資料流（stream）傳送給客戶端。

普通的 HTTP 回應有 `Content-Length` header，告訴客戶端要接收多少資料。SSE 改用 `Transfer-Encoding: chunked`，告訴客戶端回應是一系列分塊，不知道有幾塊、也不知道每塊多大，邊收邊算。這讓我們從單一的原子請求/回應，變成更細粒度的資料「串流」。

用 SSE，伺服器不是等到有資料才一次回傳完整回應，而是有資料就立刻送一塊，然後保持連線繼續送。SSE 完美適合伺服器需要持續推送資料給客戶端、但客戶端不需要頻繁回傳資料的場景。

現代瀏覽器透過 `EventSource` 物件原生支援 SSE，讓客戶端實作很簡單：

```
// 客戶端
const eventSource = new EventSource('/api/updates');

eventSource.onmessage = (event) => {
  const data = JSON.parse(event.data);
  updateUI(data);
};

// 伺服器端 (Node.js / Express)
app.get('/api/updates', (req, res) => {
  res.setHeader('Content-Type', 'text/event-stream');
  res.setHeader('Cache-Control', 'no-cache');
  res.setHeader('Connection', 'keep-alive');

  const sendUpdate = (data) => {
    res.write(`data: ${JSON.stringify(data)}\\n\\n`);
  };

  dataSource.on('update', sendUpdate);

  req.on('close', () => {
    dataSource.off('update', sendUpdate);
  });
});
```

優點

- 瀏覽器原生支援
- 自動重新連線
- 建立在 HTTP 上
- 比長輪詢更有效率，不需要反覆建立和拆除連線
- 實作簡單

缺點

- 只能單向傳輸（伺服器到客戶端）
- 部分 proxy 和網路設備不支援串流回應，這會讓偵錯變得非常麻煩。
- 瀏覽器對同一個域名的並發連線數有限制
- 因為請求可能掛很久，監控起來比較麻煩

SSE 今天最熱門的使用案例是 AI 聊天應用，這需要把產生的 token（文字）一個個串流給用戶，讓 UI 保持回應性。

面試中怎麼說

SSE 使用現有的 HTTP 基礎設施，不需要解釋太多額外的東西，也沒有輪詢間隔需要調整。

大多數 SSE 連線不會存活太久（30 到 60 秒很典型），如果你需要更長時間持續推送訊息，要討論客戶端怎麼重新建立連線，以及如何處理中斷期間的空缺。SSE 標準包含「last event ID」機制，`EventSource` 物件也明確處理重新連線邏輯，像是客戶端斷線後重連，帶上最後收到的 event ID，伺服器就能補發中斷期間遺漏的事件。

WebSocket：全雙工冠軍

WebSocket 是真正雙向通訊的首選。如果你需要高頻的讀和寫，WebSocket 是最佳解。

WebSocket 透過 HTTP 的「升級」協定（upgrade protocol）建立，它讓現有的 TCP 連線切換應用層協定。這很方便，因為你可以利用現有的 HTTP session 資訊（cookies、headers 等）。

連線建立後，客戶端和伺服器都可以互傳「訊息」，本質上是不透明的二進位資料，你可以塞 string、JSON、Protobuf 或任何東西進去。把 WebSocket 想成一個 TCP 連線，但加了一些讓連線建立更容易的貼心設計，特別是對瀏覽器來說。

```
// 客戶端
const ws = new WebSocket('ws://api.example.com/socket');

ws.onmessage = (event) => {
  const data = JSON.parse(event.data);
  handleUpdate(data);
};

ws.onclose = () => {
  // 實作重新連線邏輯
  setTimeout(connectWebSocket, 1000);
};

// 伺服器端 (Node.js / ws 套件)
const WebSocket = require('ws');
const wss = new WebSocket.Server({ port: 8080 });

wss.on('connection', (ws) => {
  ws.on('message', (message) => {
    const data = JSON.parse(message);
    processMessage(data);
  });

  dataSource.on('update', (data) => {
    ws.send(JSON.stringify(data));
  });
});
```

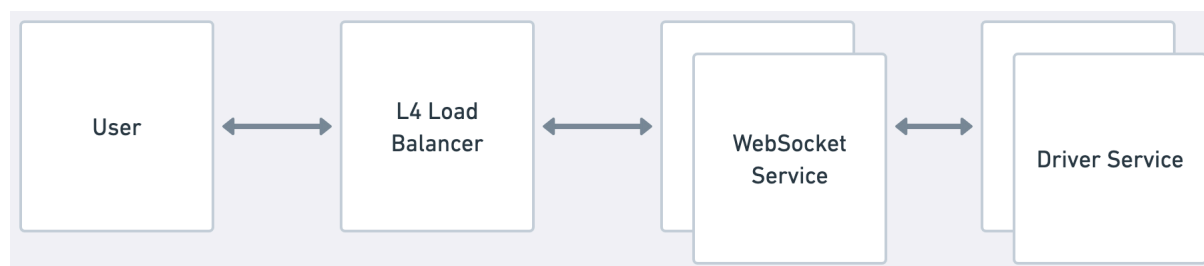
WebSocket 的額外挑戰

WebSocket 是持久連線，基礎設施必須支援它。部分 L7 負載平衡器支援 WebSocket，但整體支援參差不齊；L4 負載平衡器因為使用同一條 TCP 連線，天然支援 WebSocket。

長時間存在的連線還帶來部署問題。伺服器重新部署時，要麼切斷所有舊連線讓它們重新連，要麼讓新伺服器接管現有連線。前者更簡單，但需要考慮連線有多「持久」，以及客戶端重連時如何處理中斷期間遺漏的更新。

負載平衡更複雜，如果連線是真正長存的，一旦做出了分配決策就必須堅持。

因為有狀態連線的這些麻煩，很多架構選擇把 WebSocket 在基礎設施早期就終止到一個專門的 WebSocket 服務。這個服務處理連線管理和擴展問題，讓其他系統保持無狀態。WebSocket 服務本身比較少需要重新部署，也就減少了連線被切斷的頻率。



優點

- 全雙工（可讀可寫）通訊
- 比 HTTP 延遲更低，沒有 header 等 overhead
- 高頻訊息時很有效率
- 廣泛的瀏覽器支援

缺點

- 實作更複雜
- 需要特殊基礎設施
- 有狀態連線讓負載平衡和擴展更複雜
- 需要處理重新連線

什麼時候適合用

一般來說，如果你需要高頻的雙向通訊，就用 WebSocket。強調高頻，因為你永遠可以為寫入發出額外的請求或連線，常見的模式是用 SSE 訂閱更新，有寫入時用普通的 HTTP POST/PUT。

我常看到候選人過於急切地採用 WebSocket，其實用 SSE 或簡單輪詢就夠了。因為有額外的複雜度和基礎設施負擔，除非確實需要雙向通訊，否則先考慮 SSE。

WebRTC：點對點方案

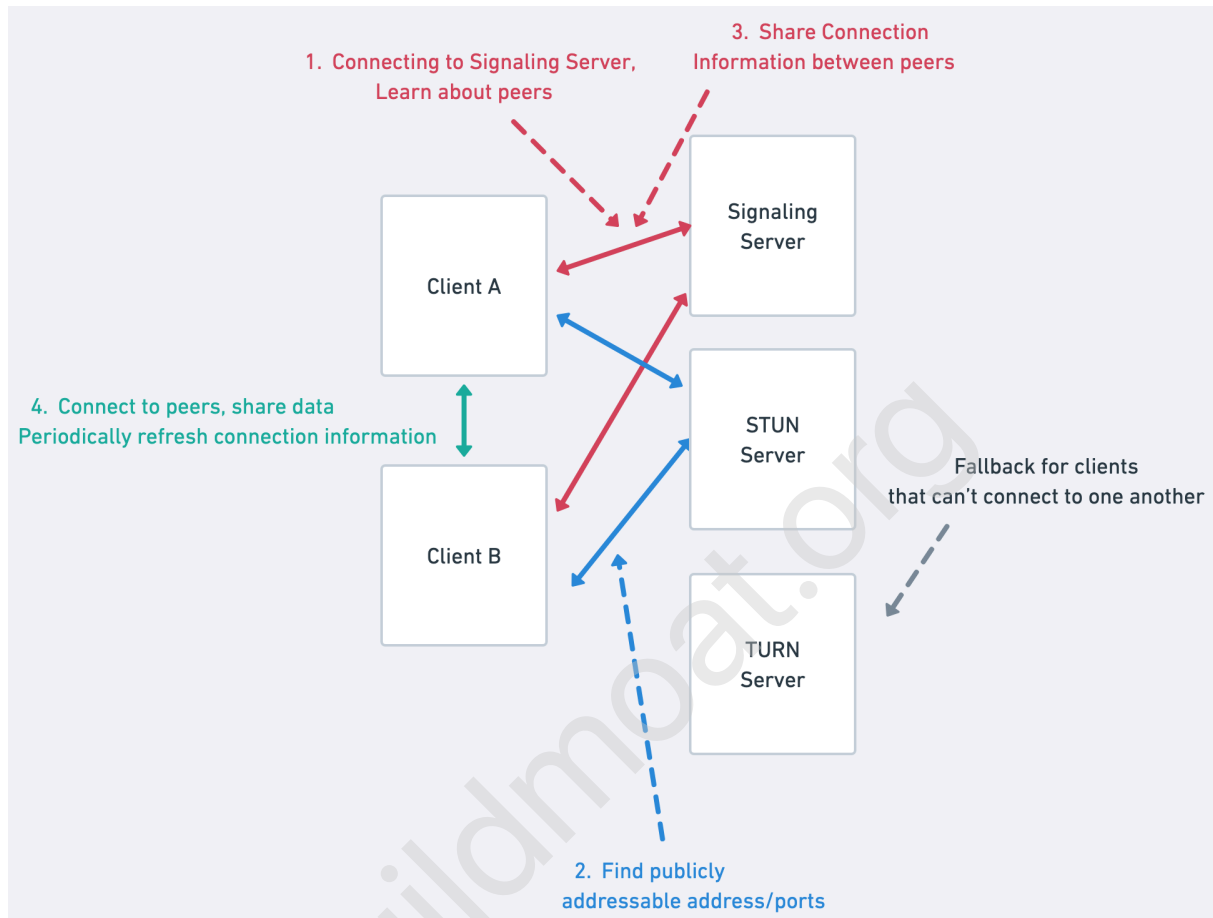
最後一個選項是最獨特的。WebRTC 讓瀏覽器之間可以直接點對點（P2P）通訊，完美適合視訊/音訊通話以及部分資料共享場景（如文件編輯器）。

客戶端透過一個中央「訊號伺服器」（signaling server）互相發現，訊號伺服器追蹤哪些 peer 可用及其連線資訊。一旦客戶端拿到另一個 peer 的連線資訊，就可以嘗試直接建立連線，不需要透過任何中間伺服器。

實際上大多數客戶端出於安全考量不允許入站連線（NAT 是主要原因）。WebRTC 標準用兩種方式繞過這個限制：

STUN (Session Traversal Utilities for NAT)：用「打洞」(hole punching) 等技術讓 peer 建立公開可路由的地址和 port，讓它們能互相找到對方。

TURN (Traversal Using Relays around NAT)：本質上是一個中繼服務，透過中央伺服器轉發請求，再路由到對應的 peer。在打洞失敗時作為 fallback。



```
// 簡化的 WebRTC 設定
async function startCall() {
  const pc = new RTCPeerConnection();

  const stream = await navigator.mediaDevices.getUserMedia({
    video: true,
    audio: true
  });

  stream.getTracks().forEach(track => {
    pc.addTrack(track, stream);
  });

  const offer = await pc.createOffer();
  await pc.setLocalDescription(offer);

  // 把 offer 送到訊號伺服器
  signalingServer.send(offer);
}
```

有趣的是，訊號伺服器本身就需要某種即時更新機制（讓 peer 能找到彼此並更新連線資訊），所以它也需要用 WebSocket、SSE 或上面介紹的其他方案之一。

優點

- 直接 peer 通訊，延遲更低
- 減少伺服器成本
- 原生音訊/視訊支援

缺點

- 設定比 WebSocket 更複雜
- 需要訊號伺服器
- NAT/防火牆問題
- 連線建立有延遲

什麼時候適合用

WebRTC 是所有方案中最複雜的，對大多數即時更新場景來說是殺雞用牛刀。但它是視訊/音訊通話、螢幕分享和遊戲的理想選擇。值得一提的例外是用它來降低伺服器負載，如果客戶端需要頻繁互相通訊，讓它們直接建立 P2P 連線可以大幅減少伺服器壓

力。Canva 就在他們的畫布編輯器中用這個方式來做游標位置同步。

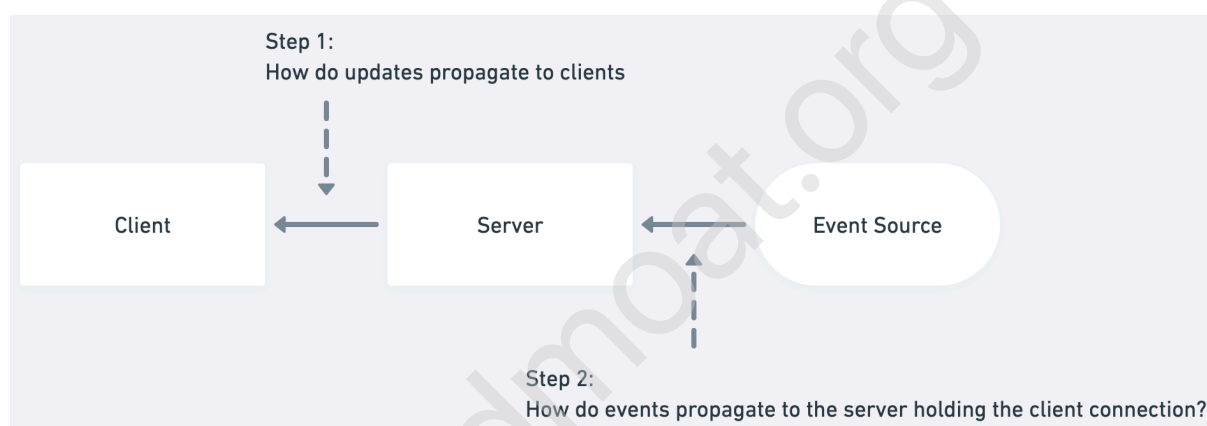
選擇指南

以下快速整理各方案的使用時機：

- 不在意延遲 → 簡單輪詢，最簡單的基準方案
- 不需要雙向通訊 → SSE，輕量且適合大多數推送場景
- 需要高頻雙向通訊 → WebSocket，複雜但效能最強
- 需要音訊/視訊通話 → WebRTC，唯一適合的選擇

第二個 Hop：伺服器端的更新傳播

處理好客戶端-伺服器通訊之後，現在來談如何把更新從事件來源傳播到伺服器。



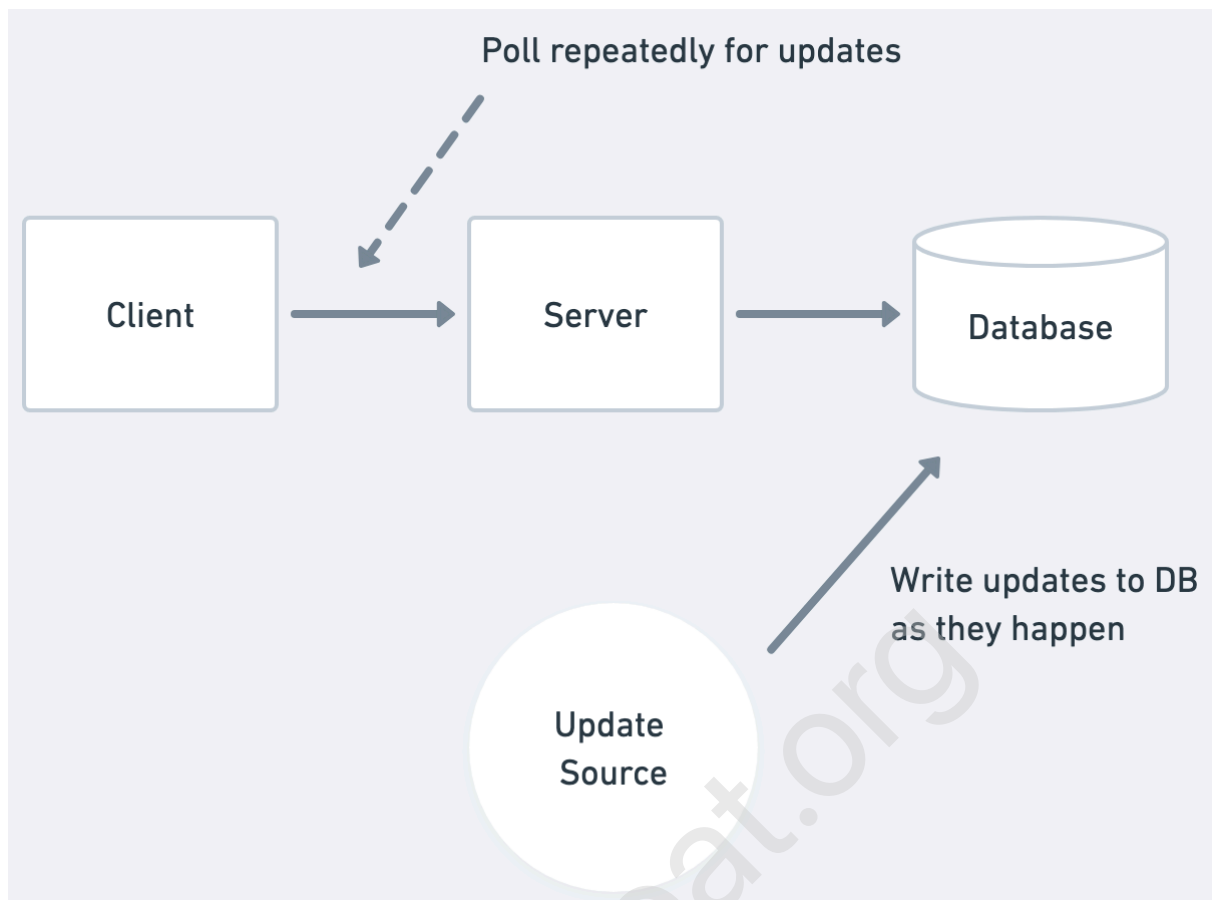
我們的系統不斷在產生需要推送給客戶端的更新，例如其他用戶對共享文件的編輯、司機的位置更新、朋友發到聊天室的訊息。確保這些更新抵達最終目的地，和我們如何從來源傳播到伺服器密切相關。換句話說，我們需要一個**觸發機制**。

常見的觸發模式有三種：

拉取：簡單輪詢

最基本的方式。客戶端不斷向伺服器請求更新，伺服器維護足夠的狀態來回應這些請求，最常見的做法是用資料庫存儲更新（例如聊天室裡的所有訊息），客戶端在需要時從資料庫拉取未接收的更新。

這個做法的美妙之處在於把更新的來源和接收方解耦了：資料庫橫亙在生產者和消費者之間，資料不是直接從生產者流向消費者的。代價是損失了即時性。



優點

- 實作極其簡單
- 狀態只集中在資料庫中
- 不需要特殊基礎設施

缺點

- 延遲高
- 更新稀少但輪詢頻繁時，資料庫負載過高
- 客戶端量很大時容易忘記計算：一百萬個客戶端每 10 秒輪詢一次，就是每秒 10 萬次讀取！

推送：一致性雜湊（Consistent Hashing）

長輪詢、SSE、WebSocket 這些方案都需要客戶端和某台伺服器保持持久連線，由那台伺服器負責把更新送給客戶端。

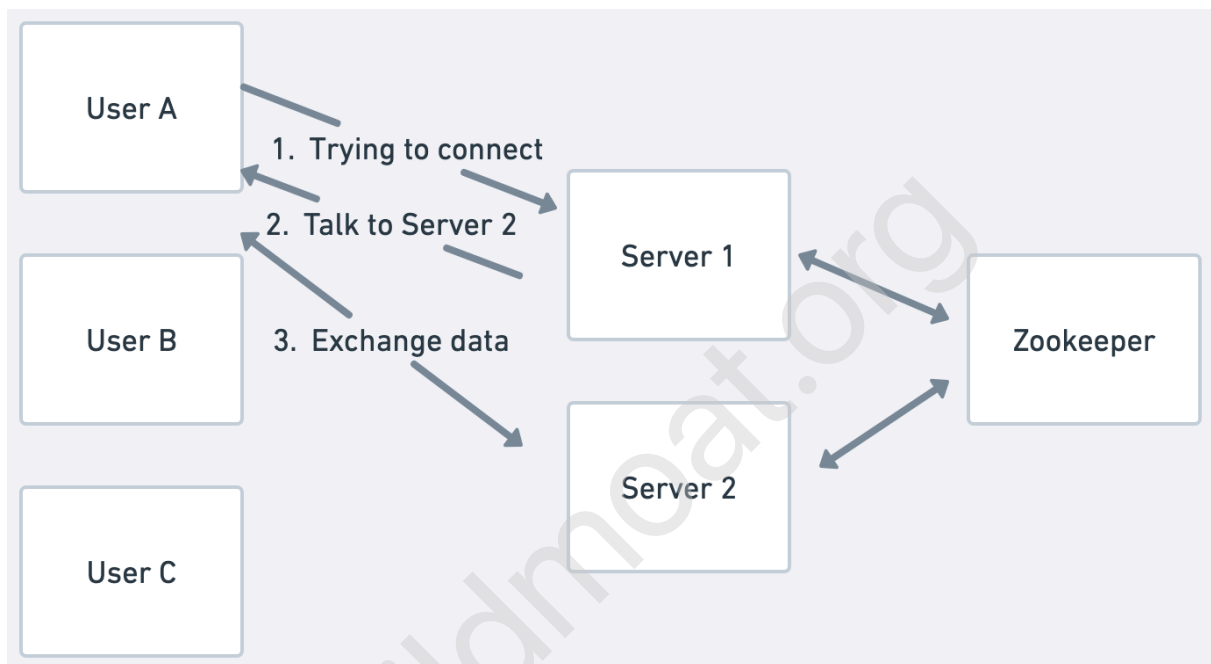
這就產生了一個問題：為了把訊息送給用戶 C，我需要知道他連線到了哪台伺服器。

簡單雜湊

第一個直覺是用取餘數（modulo）來決定哪台伺服器負責某個用戶。假設有 N 台伺服器，每台編號 0 到 $N-1$ ，讓編號為 $\text{user_id} \% N$ 的伺服器負責那個用戶的連線。這樣永遠只有一台伺服器「擁有」那個用戶的連線。

實作上通常用 Apache ZooKeeper 或 etcd 這樣的協調服務來管理 N 和伺服器的對應關係。客戶端連線時，先連到隨機的伺服器，那台伺服器查 ZooKeeper 確認應該由哪台伺服器負責，再把客戶端重導向到正確的伺服器。

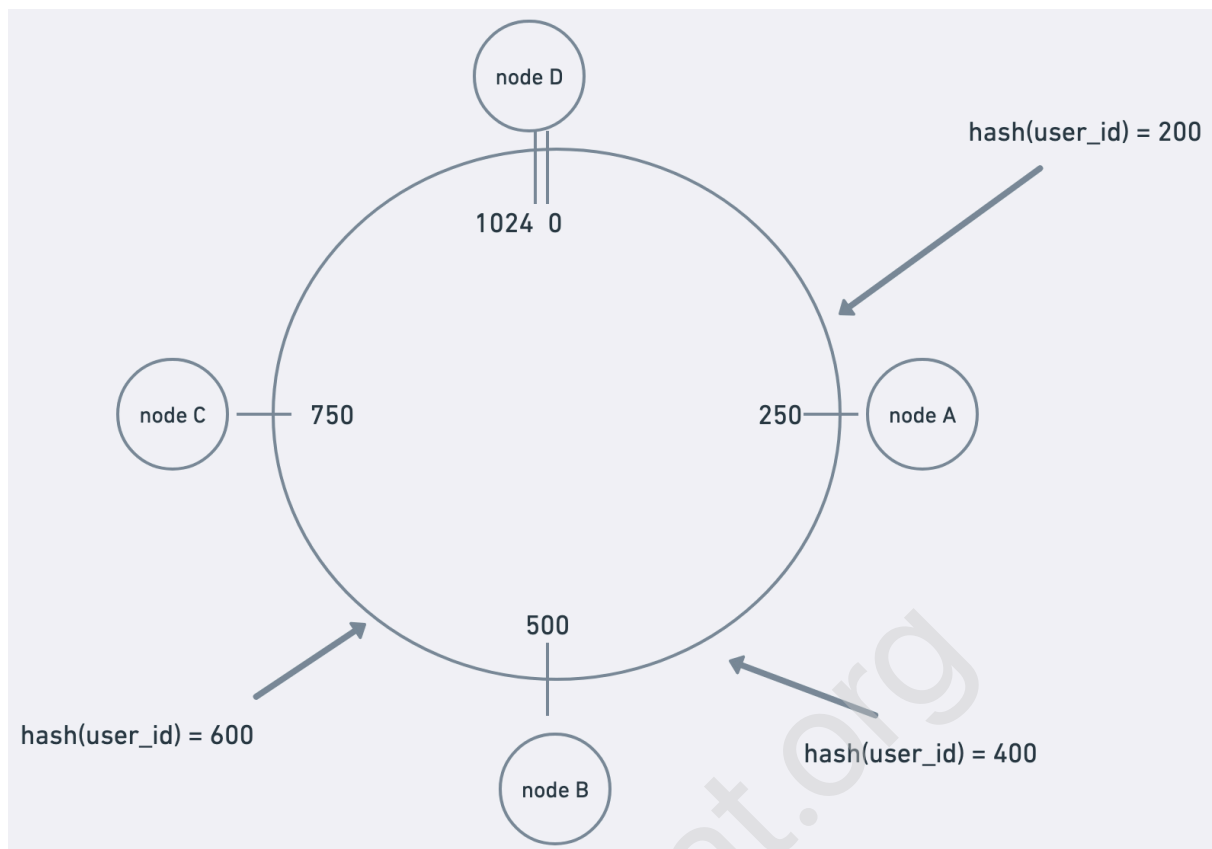
要把訊息傳給用戶 C 時，更新服務就對用戶 ID 做雜湊，找到負責的伺服器，把訊息發過去，那台伺服器再透過對應的連線轉發給用戶。



一致性雜湊（Consistent Hashing）

簡單雜湊在 N 固定時運作良好，但需要增減伺服器時，就會變得非常麻煩，因為改變 N 幾乎會導致所有用戶都要斷開重連到不同的伺服器。

一致性雜湊透過把伺服器和用戶都映射到一個雜湊環（hash ring）上來解決這個問題。每個用戶沿著環順時針方向連到遇到的第一台伺服器。新增或移除伺服器時，只有環上受影響段落的用戶需要遷移，大幅減少了擴縮容時的連線動盪。



優點

- 伺服器分配可預測
- 擴縮容時連線遷移最小化
- 適合有狀態連線
- 容易新增/移除伺服器

缺點

- 實作正確有一定難度
- 需要協調服務（如 ZooKeeper）
- 所有伺服器需要維護路由資訊
- 伺服器失效時連線狀態遺失

適合場景

一致性雜湊適合需要維護持久連線（WebSocket/SSE）且系統需要動態擴縮的場景，特別是每個連線需要維護大量狀態的情況，例如 Google Docs 設計中，連線和特定文件關聯，維護協作編輯功能需要大量狀態。

如果你只是在向客戶端傳遞小訊息，沒有太多關聯狀態，用下面的 Pub/Sub 方案會更好。

面試中怎麼說

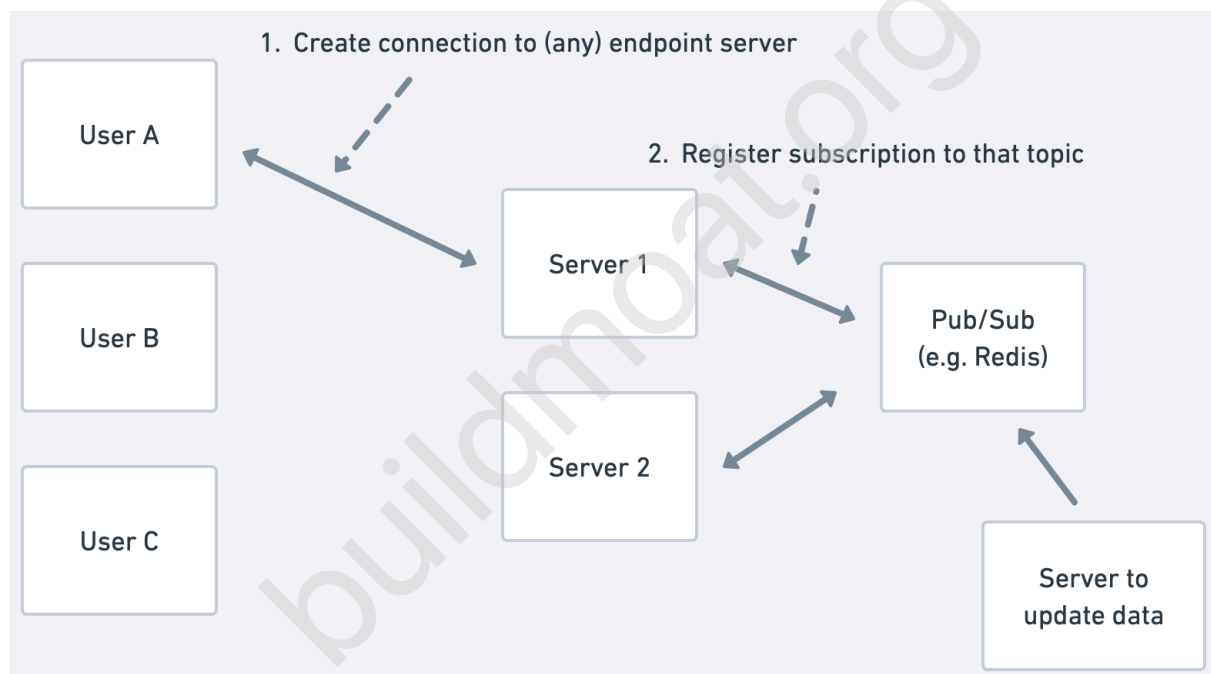
引入一致性雜湊時，要能說明更新如何被路由（例如透過 ZooKeeper 或 etcd 這樣的協調服務），以及擴縮容時系統怎麼運作。實際操作通常包括：

6. 發出擴縮容開始的訊號，同時記錄舊的和新的伺服器分配
7. 慢慢斷開客戶端到舊伺服器的連線，讓它們重新連到新分配的伺服器
8. 發出擴縮容結束的訊號，更新協調服務的伺服器分配
9. 過渡期間，訊息同時發往舊伺服器和新伺服器

推送：Pub/Sub

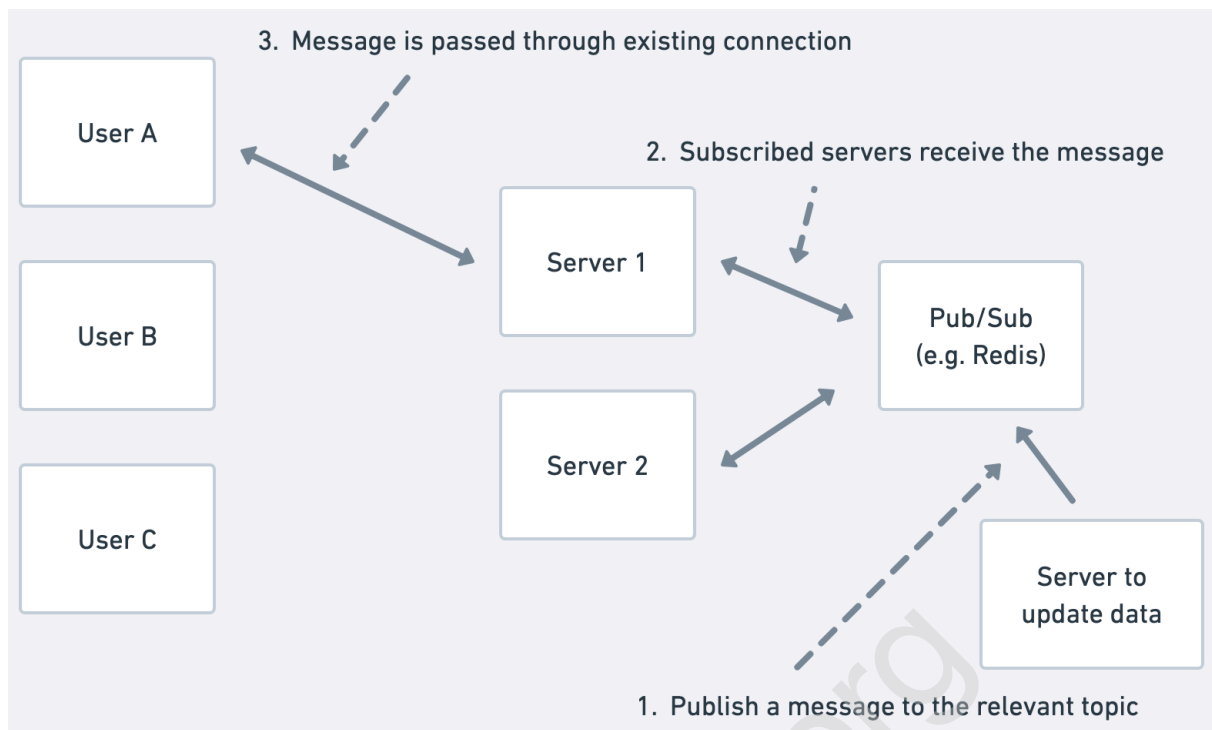
另一個觸發方式是 Pub/Sub 模型。有一個專門的服務負責收集來源的更新，然後廣播給所有感興趣的客戶端。常見的選項包括 Kafka 和 Redis。

Pub/Sub 服務成為即時更新最主要的狀態儲存點。持久連線現在連到輕量的「端點伺服器」，這些伺服器只需訂閱相關的 topic，把更新轉發給對應的客戶端。



連線建立時：

10. 客戶端連線到任意一台端點伺服器（不需要像一致性雜湊那樣連到特定伺服器）
11. 端點伺服器向 Pub/Sub 服務注冊客戶端，通常是建立一個 topic、訂閱它，並維護一個從 topic 到對應連線的映射



更新廣播時：

12. 更新被推送到 Pub/Sub 服務的對應 topic
13. Pub/Sub 服務把更新廣播到所有訂閱了那個 topic 的伺服器
14. 端點伺服器收到更新，查找哪個客戶端訂閱了那個 topic，透過現有連線把更新轉發出去

以聊天 App 為例：為每個用戶建立一個 topic，客戶端連線到端點伺服器時，訂閱對應用戶的 topic。要傳送訊息時，把它發布到目標用戶的 topic，Pub/Sub 服務廣播到所有訂閱的伺服器，再轉發給客戶端。

優點

- 端點伺服器的負載平衡很容易，像是用「最少連線數」（least connections）策略的負載平衡器就好
- 可以有效率地廣播給大量客戶端
- 最小化狀態在系統中的擴散

缺點

- 不知道訂閱者什麼時候連線到端點伺服器，也不知道什麼時候斷開
- Pub/Sub 服務是單點故障和瓶頸
- 增加了一層間接層，可能增加延遲
- Pub/Sub 服務主機和端點伺服器之間存在多對多連線

什麼時候適合用

Pub/Sub 是需要廣播更新給大量客戶端時的好選擇。設定簡單，端點伺服器的 overhead 很少，延遲影響很小 ($< 10\text{ms}$)。如果不需要響應連線/斷線事件、也不需要維護大量與客戶端相關的狀態，這是個好選擇。

面試中怎麼說

用 Pub/Sub 時，可能需要討論它作為單點故障的問題。Redis Cluster 是擴展 Pub/Sub 服務的熱門方式——把訂閱按 key 分片到多台主機上，擴展支援的訂閱數量和吞吐量。

引入 cluster 意味著你要管理 Pub/Sub 服務和端點伺服器之間的多對多連線（每台端點伺服器都連接到 cluster 的所有主機）。在很多情況下，cluster 節點數很少，這個問題還算可控。

對於入站連線，端點伺服器前面用「最少連線數」策略的負載平衡器，可以確保負載均勻分散。因為連線本身（和透過它傳送的訊息）是被消耗的主要資源，基於連線數來做負載平衡是管理負載的好方法。

什麼時候在面試裡用這些

即時更新幾乎出現在每個涉及用戶互動或即時資料的系統設計面試中。不要等面試官來問即時功能，主動辨識哪些地方需要立即更新，在你的初始設計中就提出來。

強候選人在早期就識別即時更新的需求。設計聊天應用時，直接說「訊息需要立刻送達所有參與者，我用 WebSocket 處理這個雙向通訊」。設計協作編輯時，提出「字元層級的改動需要在用戶之間做毫秒級傳播」。

常見面試情境

聊天應用：最經典的即時使用案例。訊息必須在所有參與者之間即時顯示。

WebSocket 完美處理雙向通訊，Pub/Sub 把訊息分發到正確的伺服器。要考慮訊息排序、正在輸入的提示 (typing indicator) 和在線狀態 (presence)。

直播留言：直播活動期間大量、即時的社交互動。幾百萬觀眾同時留言製造了極端的扇出 (fan-out) 問題。階層式聚合和仔細的批次處理可以防止系統過載，同時維持即時感。

協作文件編輯：字元層級的改動需要在用戶之間即時傳播。WebSocket 提供低延遲通訊，Operational Transform 或 CRDT 處理衝突解決問題。用戶游標和選取範圍的高亮顯示又增加了額外的即時複雜度。

協作編輯不只需要低延遲高頻率地把更新送達客戶端，還需要處理衝突、確保文件狀態一致，如果用戶可以同時在同一個地方輸入，就需要能處理衝突。Google Docs 的設計中有兩種方法：CRDT 和 Operational Transform。

即時儀表板和分析：不斷變化的業務指標和營運資料。SSE 非常適合從伺服器到儀表板的單向資料流。要考慮資料聚合間隔，以及對業務決策來說「夠即時」是什麼程度。

遊戲和互動應用：多人遊戲需要最低的延遲。WebRTC 讓點對點通訊降低延遲，WebSocket 處理伺服器協調。要考慮不同遊戲元素的不同更新頻率。

什麼時候不適合用

在能用簡單輪詢解決問題的時候，就避免引入即時更新。如果不在意延遲，輪詢是極佳的基準方案，讓複雜度降到最低，這是 Senior+ 面試中非常被欣賞的特質。用輪詢可以同時迴避兩個 hop：不需要煩惱客戶端-伺服器協定，也不需要處理從事件來源的傳播。

常見的 Deep Dive 問題

「連線失敗和重新連線怎麼處理？」

真實世界的網路是不可靠的。行動用戶一直在斷線，WiFi 時常掉線，伺服器也會重啟。你的即時系統需要優雅的降級和恢復機制。

核心挑戰是快速偵測斷線並在不遺失資料的情況下恢復。WebSocket 連線不一定會發出斷線訊號，客戶端可能以為自己還連著，但伺服器已經清掉了那條連線。實作心跳機制（heartbeat）可以幫助偵測這種「殭屍連線」。

對於恢復，你需要追蹤客戶端已收到哪些訊息或更新。重連時，它應該收到所有它錯過的內容。這通常意味著維護一個按用戶的訊息 queue，或實作序列號讓客戶端在重連時可以引用。用 Redis Stream 來做這個是一個熱門方案。

「某個用戶有幾百萬個追蹤者，他們全都需要同一個更新，怎麼辦？」

這是即時系統中經典的「名人問題」。當名人發文，幾百萬個用戶需要立刻收到那個更新。天真的做法製造的巨大扇出（fan-out）可能讓你的系統崩潰。

解法是策略性的快取和階層式分發。不是把更新寫入幾百萬個個別用戶的 feed，而是只快取一次，透過多個層次分發出去。各地區的伺服器可以拉取更新並推送給本地客戶端，降低任何單一元件的負載。詳見寫入擴展講義中關於批次處理和階層式聚合的部分。

「如何在分散式伺服器之間維護訊息順序？」

當多台伺服器處理即時更新，確保一致的順序變得複雜。兩條訊息相隔幾毫秒發出，如果它們走不同的網路路徑或被不同的伺服器處理，可能會亂序到達。

Vector clock 或邏輯時間戳記可以幫助建立訊息之間的順序關係。每台伺服器維護自己的時鐘，訊息帶上時間戳記資訊，讓接收方能決定正確的順序。

對於嚴格的順序要求，你可能需要讓所有相關訊息都流經同一台伺服器或同一個 partition。這以犧牲部分擴展性換來一致性保證，但大幅簡化了排序問題。

對大多數產品導向的系統設計面試而言，讓所有訊息通過單一主機或 partition 是最實際的做法。Vector clock 等技術更多地出現在基礎設施深度問題中，而不是「設計一個線上拍賣系統」這類題目。如果所有訊息都流過同一台主機，打上正確的時間戳記並建立全序（total order）就很直觀。

總結

即時更新是系統設計中最有挑戰性的模式之一，幾乎出現在每個互動式應用中，從聊天到協作編輯。核心洞察是即時系統需要解決兩個截然不同的問題：客戶端-伺服器通訊協定，以及伺服器端的更新傳播。

從簡單開始，根據需求升級。 如果每幾秒輪詢一次就能滿足需求，不要跳到複雜的 WebSocket 架構。大多數候選人會過度設計即時方案，其實更簡單的做法就夠了。然而當真正的即時效能確實被需要時，理解各協定之間的取舍就至關重要。

客戶端通訊：SSE 和 WebSocket 能有效處理大多數即時場景。SSE 對像即時儀表板這樣的單向更新效果很好，WebSocket 在需要雙向通訊時表現最佳。

伺服器端：Pub/Sub 系統為大多數應用提供了最佳的簡單性和可擴展性的平衡點。它把更新來源和客戶端連線解耦，讓系統更容易推理和擴展。當連線狀態管理成為主要問題時，才考慮一致性雜湊的方案。

即時更新應用帶來了很多有趣的複雜度：低延遲、擴展性、網路問題，以及管理多個服務和狀態的需求。掌握這些選項的取舍，你就能在面試中做出最佳的設計決策並清楚地傳達你的推理。